

Assignment 1

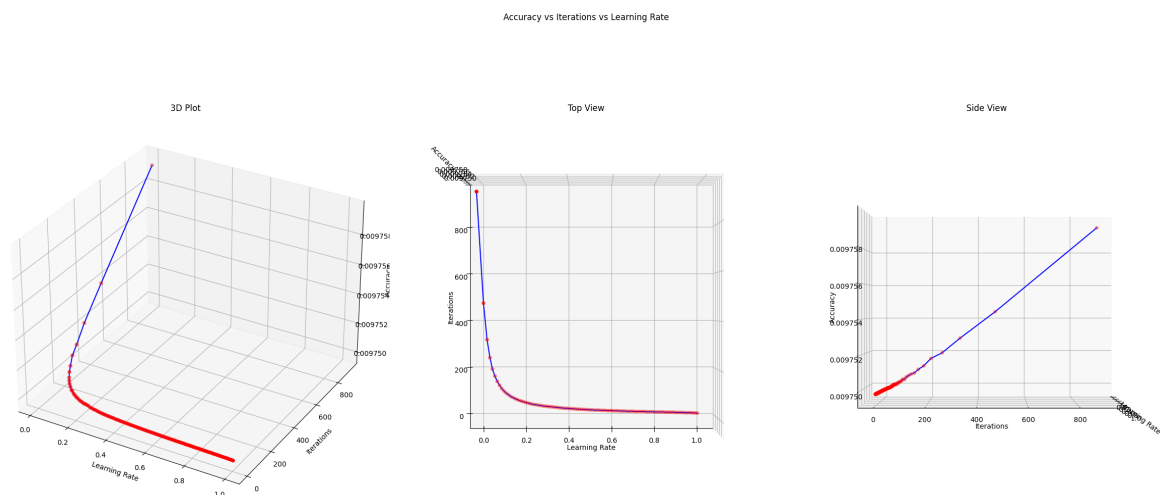
Total Points : 84

Due Date: 11:59 pm, Friday, Feb 14, 2025

Umang Tripathi (2022CS51134)

1. (20 points) Linear Regression

1.1 (8 points)

(a) Implement batch gradient descent method for optimizing $j(\theta)$ **Answer:** Implemented batch gradient descent in linear_regression.py in Q1 folder(b) Choose (and report) an appropriate learning rate and the stopping criteria (as a function of the change in the value of $J(\theta)$). You can initialize the parameters as $\theta = \vec{0}$ (the vector of all zeros). Do not forget to include the intercept term.**Answer:** Stopping criteria was set to be when if $\text{abs}(J_theta - \text{prev_}J_theta) < 1e-9$: break
Where J_theta is value of $J(\theta)$ on the current iteration and $\text{prev_}J_theta$ is value of $J(\theta)$ in the previous iteration. Another stopping criteria was set to number of iteration being $1e6$ just so that my algorithm stops after some finite time, for the data given this condition was never reached.

In the above graph (here I measured accuracy as MSE)

(c) Report your learning rate, stopping criteria and the final set of parameters obtained by your algorithm

Answer :

Learning rate	$\theta = [\theta_0, \theta_1]$	MSE	Iterations
0.001	$\theta = [6.21660435 \ 29.05498543]$	0.00984954267	8002

0.025	$\theta = [6.21828585 \ 29.06290425]$	0.00975346676	382
0.1	$\theta = [6.21851283 \ 29.06397453]$	0.00975052660	100
1.0	$\theta = [6.21867801 \ 29.06475515]$	0.00974989055	4

I choose my learning rate as 1.0

My stopping criteria as when my previous cost and current cost differ by less than $1e-7$

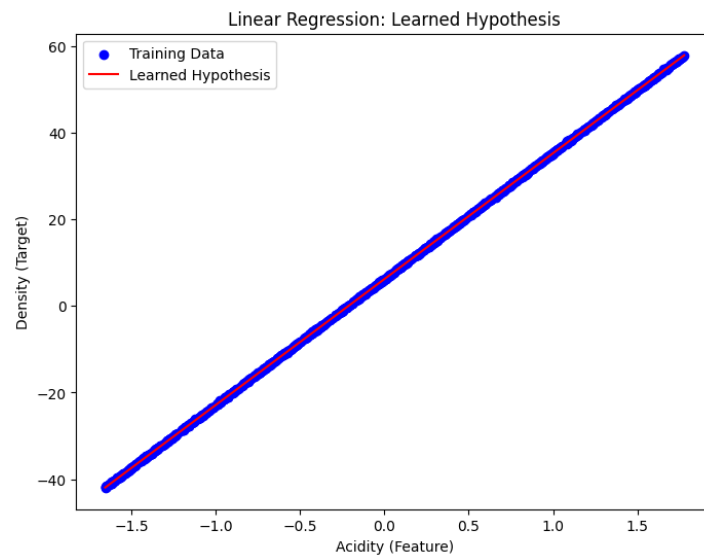
And the final set of parameters I report are :

$$\theta = [6.21867801, 29.06475515]$$

1.2. (3 points)

Plot the data on a two-dimensional graph and plot the hypothesis function learned by your algorithm in the previous part.

Answer :



1.3. (3 points)

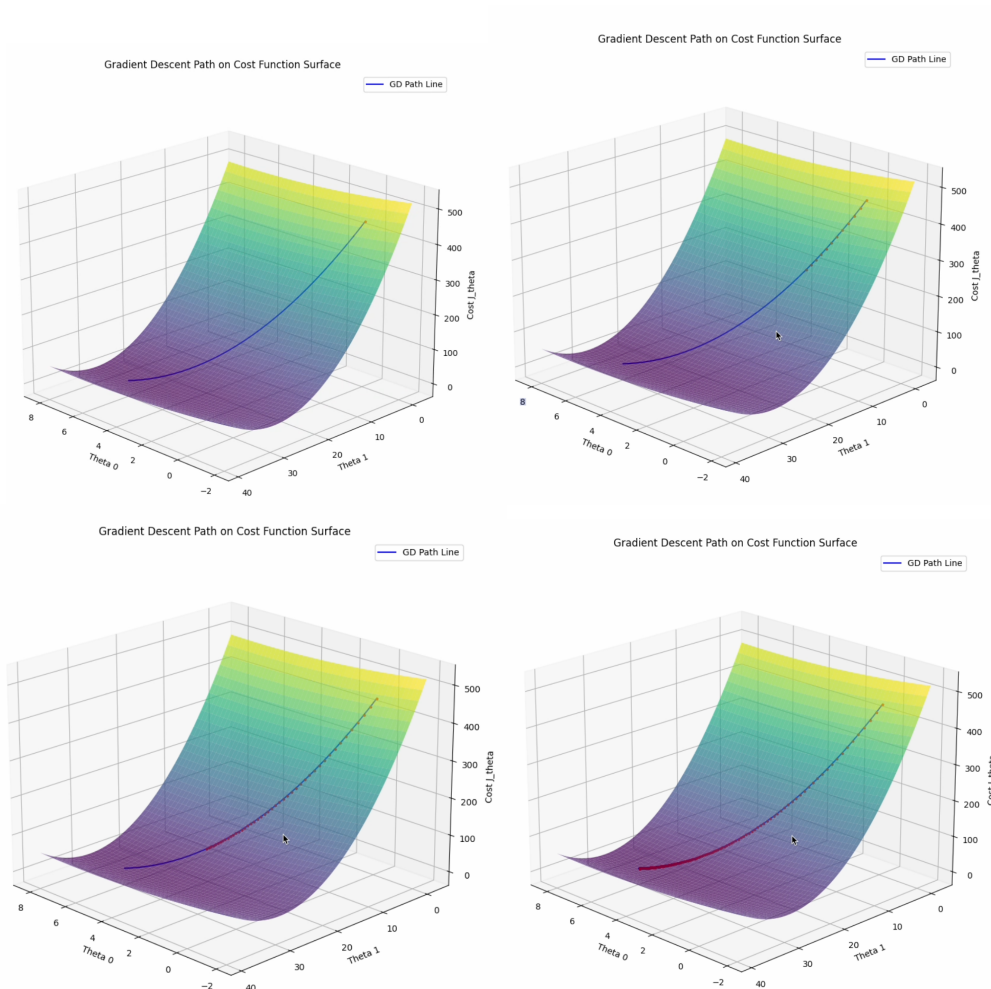
(a) Draw a 3-dimensional mesh showing the error function ($J(\theta)$) on z axis and the parameters in the x-y plane. Use a colour gradient for different values of the error function.

(b) On the same plot, display the error value using the current set of parameters at each iteration of the gradient descent. Include a time gap of 0.2 seconds in your display for each iteration so that the change in the function value can be observed by the human eye.

Answer :

Visualization can be found here (at learning rate 0.025)

https://youtu.be/fU9bAChvM_A



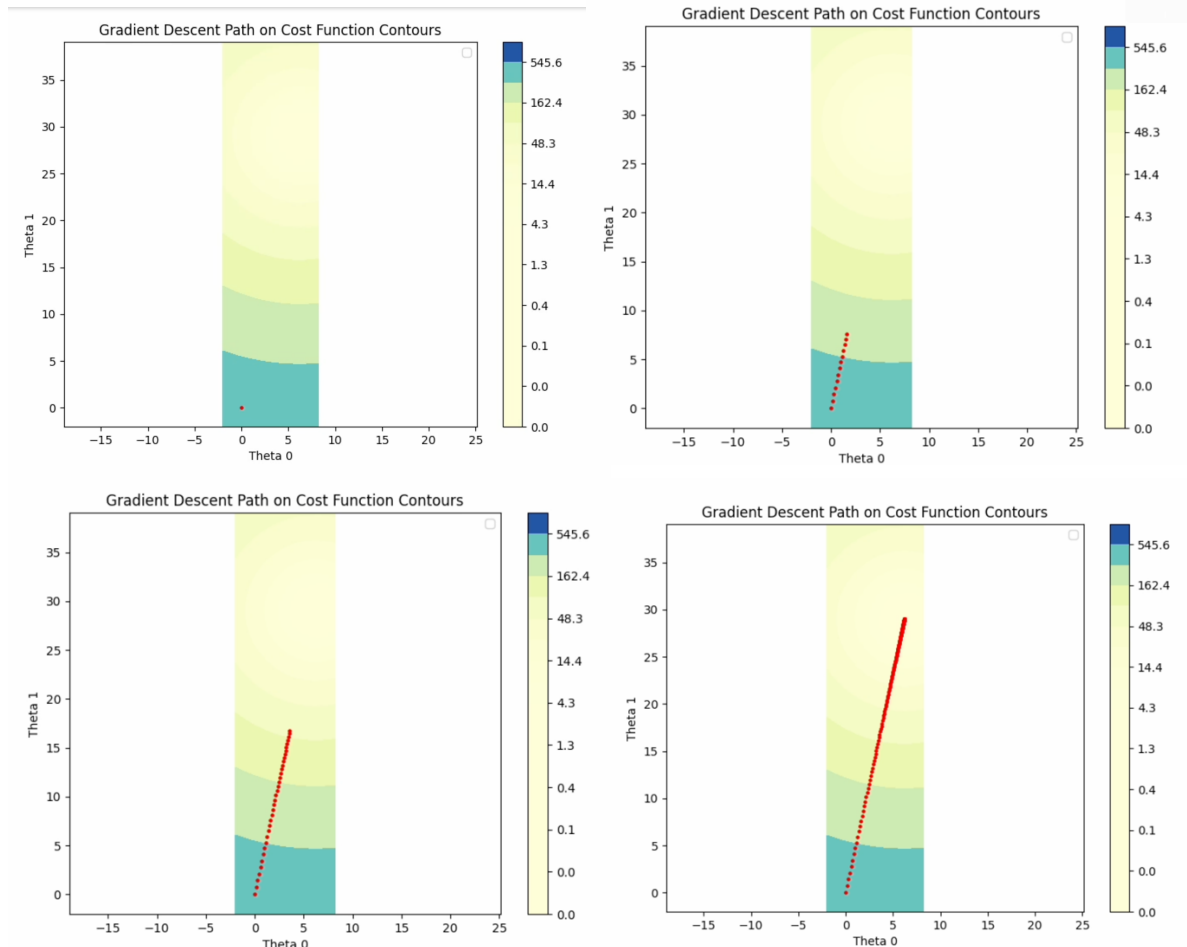
1.4. (3 points)

Repeat the part above for drawing the contours of the error function at each iteration of the gradient descent. Once again, chose a time gap of 0.2 seconds so that the change be perceived by the human eye.(Note here plot will be 2-D)

Answer :

Visualization can be found here (at learning rate 0.025)

<https://youtu.be/8eZn1JSA1W4>

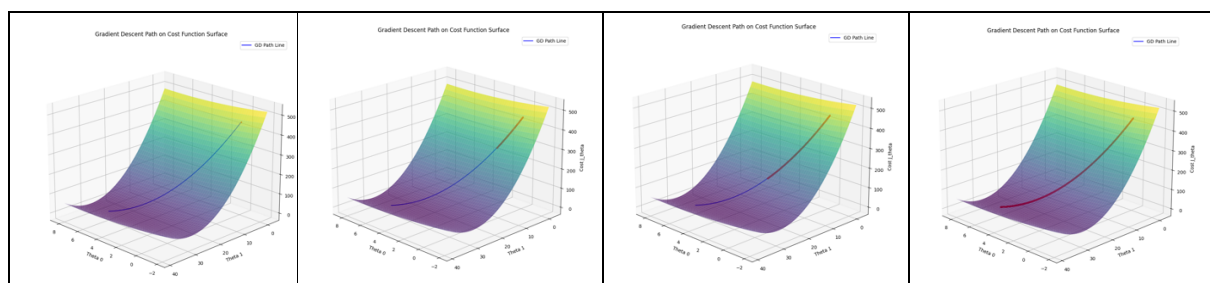


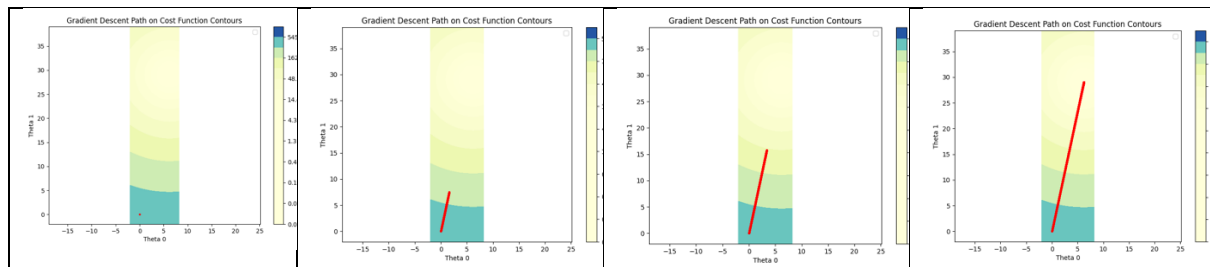
1.5. (3 points)

Repeat the part above (i.e. draw the contours at each learning iteration) for the learning rate values of $\eta = \{0.001, 0.025, 0.1\}$. What do you observe? Comment.

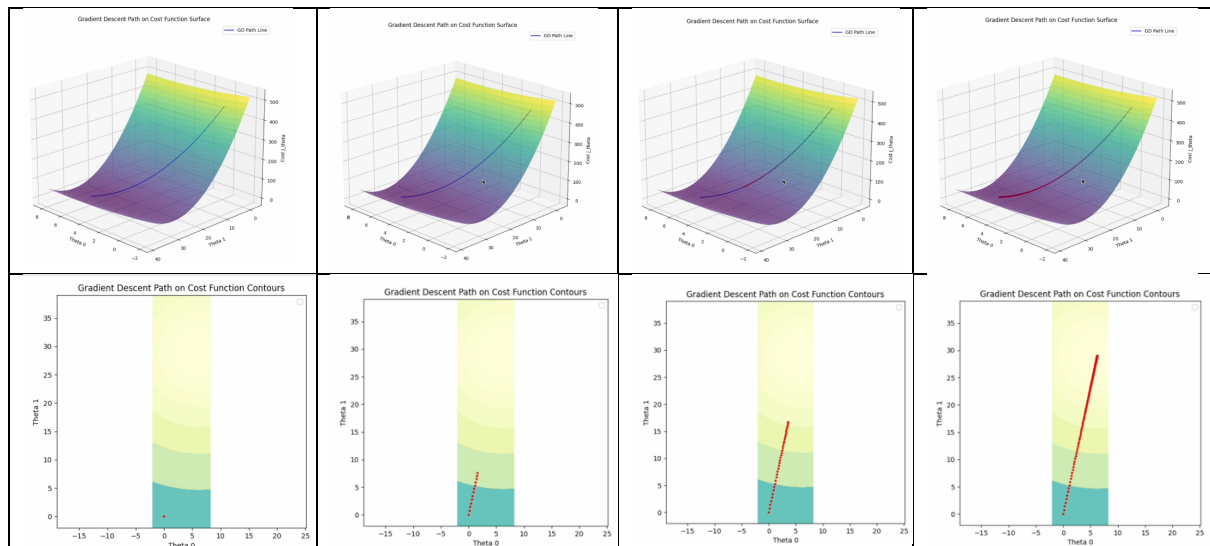
Answer :

$$\eta = 0.001$$

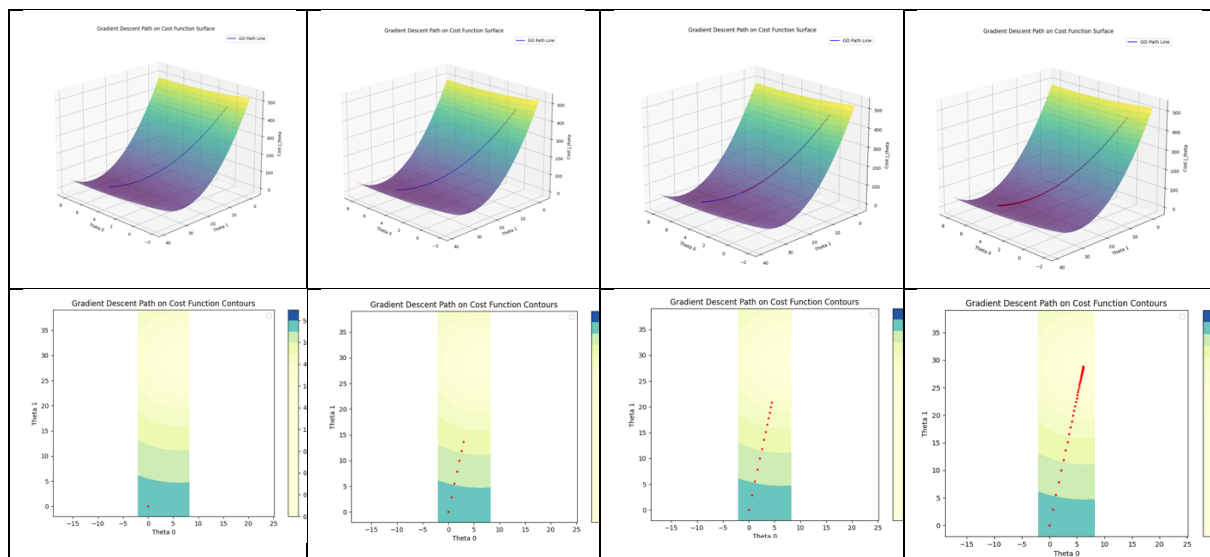




$$\eta = 0.025$$



$$\eta = 0.1$$



Effect of a High Learning Rate ($\alpha=0.1$)

- A relatively high learning rate, such as 0.1, allows the model to converge faster because it updates the parameters with larger steps.

- However, if the learning rate is too high, the model may overshoot the optimal solution, leading to divergence instead of convergence. This occurs because the parameter updates are too aggressive, preventing the loss function from settling at a minimum.
- In some cases, a high learning rate may cause the loss to oscillate around the optimal value without stabilizing.

Effect of a Moderate Learning Rate ($\alpha=0.025$ \alpha = 0.025 $\alpha=0.025$)

- A moderate learning rate, such as 0.025, provides stable convergence, meaning the loss function decreases smoothly toward its minimum.
- While this value is slower compared to 0.1, it ensures that the gradient descent process remains controlled, reducing the risk of overshooting or instability.
- This choice is typically safer for general datasets, as it maintains a balance between speed and stability.

Effect of a Very Low Learning Rate ($\alpha=0.001$ \alpha = 0.001 $\alpha=0.001$)

- A very low learning rate, such as 0.001, updates parameters with very small steps.
- While this helps in avoiding divergence, the convergence becomes extremely slow, making it impractical for most real-world applications.
- In extreme cases, the algorithm may take an excessive number of iterations to reach the optimal parameters, or it may get stuck in a region where progress is too slow to be useful.

2. (24 points) Sampling, Closed Form and Stochastic Gradient Descent

2.1. (4 points)

Sample 1 million data points taking values of $\theta = [\theta_0, \theta_1, \theta_2] = [3, 1, 2]$, $x_1 \sim N(3, 4)$ and $x_2 \sim N(-1, 4)$ independently, and noise variance in y , $\sigma^2 = 2$. Split this data into two parts - keep 80% of the data (800,000 data points) as train set, and the remaining 20% of the data (200,000 data points) as the training set.

2.2. (6 points)

(a) Implement Stochastic gradient descent method for optimizing $J(\theta)$.

(b) Relearn using the training set of part a) keeping everything same except the batch size. Keep $\eta = 0.001$ and initialize $\forall j \theta_j = 0$. Report the θ learned each time separately for values of batch size $r = \{1, 80, 8000, 800000\}$. Do not normalize any part of the data.

Answer :

Learning rate : 0.001

Batch size = $\{1, 80, 8000, 800000\}$

Theta was initialized to $[0, 0, 0]$

Batch size	θ	Epoch
1	[3.00455481 , 1.03798659 , 2.01710997]	10
80	[2.99445371 , 0.99818504 , 2.0020426]	100
8000	[2.9947801 , 1.0020084 , 1.99856346]	1000
800000	[2.81272822 , 1.04177286 , 1.98535165]	10000

(c) Carefully decide your convergence criteria in each case. Make sure to go through the material on the convergence of SGD algorithm discussed in class.

Answer :

We maintain a rolling mean of the last 10 loss values, denoted by R_t , where each R_t includes the current iteration's loss J_{θ_t} . After every update, we compute the difference $|R_{t+1} - R_t|$. If this difference remains below a specified tolerance for a certain number of consecutive iterations (iii), we conclude that the algorithm has converged. Both the tolerance and the required number of consecutive iterations vary with the batch size, based on in-class discussions. Specifically:

- Batch size = 1:
 - Tolerance: $1e-4$
 - $i=3$ consecutive iterations
- Batch size = 80:
 - Tolerance: $1e-4$
 - $i=5$ consecutive iterations
- Batch size = 8000:

- Tolerance: $1e-6$
- $i=5$ consecutive iterations
- Batch size = 800000:
 - Tolerance: $1e-7$
 - $i=5$ consecutive iterations

Formally, the convergence condition can be written as:

$$|R_{t+1} - R_t| < \text{Tolerance for } i \text{ consecutive iterations}$$

2.3. (6 points)

(a) Do different algorithms in the part above (for varying values of r) converge to the same parameter values? Comment on the relative speed of convergence and also on number of iterations in each case.

Answer :

Batch Size	Iterations	Time taken	Epochs
1	8000000	30.2 sec	10
80	1000000	12.4 sec	100
8000	100000	101.3 sec	1000
80000	10000	784.3 sec	10000

1. Small Batch Size ($r = 1$)
 - Takes the most iterations (8,000,000 updates) but each update is computationally cheap.
 - Since it makes frequent noisy updates, it fluctuates more but still converges in a reasonable time (30.2 sec).
 - Slow convergence in terms of iterations but relatively fast wall-clock time.
2. Moderate Batch Size ($r = 80$)
 - Fastest convergence in wall-clock time (12.4 sec).
 - Balances frequent updates with better gradient estimates.
 - Fewer iterations (1,000,000) than $r = 1$, while keeping updates efficient.
3. Large Batch Size ($r = 8000$)
 - Takes much fewer iterations (100,000 updates), but each iteration is computationally expensive.
 - Despite fewer updates, total training time is 101.3 sec, longer than smaller batches.
 - Converges smoothly but takes significantly more wall-clock time.
4. Very Large Batch Size ($r = 80,000$)
 - Requires the fewest updates (10,000 iterations), but each update is extremely expensive.
 - This leads to the longest training time (784.3 sec), making it the slowest method.
 - Although each update is highly accurate, the overall training speed suffers.

Key Takeaways

- Moderate batch sizes ($r = 80$) lead to the fastest overall convergence due to a balance between update frequency and computational cost.

- Smaller batches ($r = 1$) converge in fewer epochs but require many noisy updates, making them less efficient in terms of iteration count.
- Larger batches ($r = 8000, 80,000$) require fewer updates but take longer per iteration, making them impractical for fast training.
- An optimal batch size balances iteration count and computation cost—here, $r = 80$ is the fastest in terms of wall-clock time.

(b) We know that a closed form solution exists for Linear Regression, given by:

$$\theta = (X^T X)^{-1} X^T Y$$

Relearn $\theta = [\theta_0, \theta_1, \theta_2]$ using sampled data points of part a) from this closed form.

Answer :

For the data generated from $\theta = [\theta_0, \theta_1, \theta_2] = [3, 1, 2]$, $x_1 \sim N(3, 4)$ and $x_2 \sim N(-1, 4)$, The relearned value of θ found is :

$$\theta = [2.99590168, 1.00173919, 1.99873391]$$

This value is very close to the original value of $[3, 1, 2]$

(c) Compare the true parameters (from which we sampled), the parameters learned in closed form, and the parameters learnt by SGD. Write what you observe.

Answer :

1) Close to the True Parameters

- For small to moderate batch sizes ($r=1, 80, 8000$), the final theta values are very close to $[3, 1, 2]$.
- The closed-form solution—which directly solves the normal equations—also lands near $[3, 1, 2]$.

2) Large Batch Size Convergence

- With $r=80000$ (i.e., the entire training set in one batch), the first component theta is about 2.81, which is noticeably farther from 3 compared to other results.
- Likely reasons:
 - One update per epoch can slow convergence significantly.
 - The learning rate or number of epochs may not have been sufficiently tuned to allow the solution to get closer to $[3, 1, 2]$.

3) Overall Performance

- Despite slight differences in theta, all models end up with mean squared error (MSE) near the noise variance ($\sigma^2=2$), indicating that each solution fits the data well.
- There is no sign of overfitting, and training vs. test MSE remains similar.

4) Why the Closed-Form is a “Gold Standard”

- The normal equation typically gives the exact least-squares solution (assuming no major numerical issues).
- Stochastic methods like SGD can converge very close to this solution with enough updates and proper hyperparameter tuning.

2.4. (4 points)

Find the mean squared error on the test set (the 20% portion kept in part(a)) called the test error. Compare with the error on the training set called training error.

Answer :

Batch size	θ	Train MSE	Test MSE
1	[3.00455481 , 1.03798659 , 2.01710997]	2.016078	2.012141
80	[2.99445371 , 0.99818504 , 2.0020426]	1.999987	1.994527
8000	[2.9947801 , 1.0020084 , 1.99856346]	1.999656	1.994412
800000	[2.81272822 , 1.04177286 , 1.98535165]	2.009259	2.003961
Closed form	[2.99590168 , 1.00173919 , 1.99873391]	1.999655	1.994409

The table shows that all methods achieve an MSE close to 2.0, which is roughly the noise level in the data. Additionally, the training MSE and test MSE are almost the same across the board, indicating little to no overfitting.

Key Observations

- 1) All Methods Are Close to the True Parameters
- 2) Training MSE $\approx$ Test MSE . This indicates no substantial overfitting—the model generalizes well.
- 3) MSE $\approx$ 2.0 Matches Noise Variance

Since we sampled data with $\sigma^2=2$, an MSE near 2.0 is expected for a well-fitted model. This consistency confirms the algorithms are converging to a near-optimal solution.

4) Closed-Form vs. SGD

The closed-form (normal equation) solution generally yields the most accurate estimate, as expected, and has the lowest or near-lowest MSE. However, SGD solutions for moderate batch sizes (80 or 8000) come extremely close to the closed-form result.

5) Large Batch ($r = 800000$) is Slightly Off

The learned parameters [2.8127, 1.0418, 1.9854] deviate more from [3, 1, 2] than the smaller-batch or closed-form solutions. The MSE is still near 2.0, but it's a bit higher. Likely, fewer updates per epoch (only 1 update per epoch) required more careful tuning of learning rate or more epochs to converge precisely.

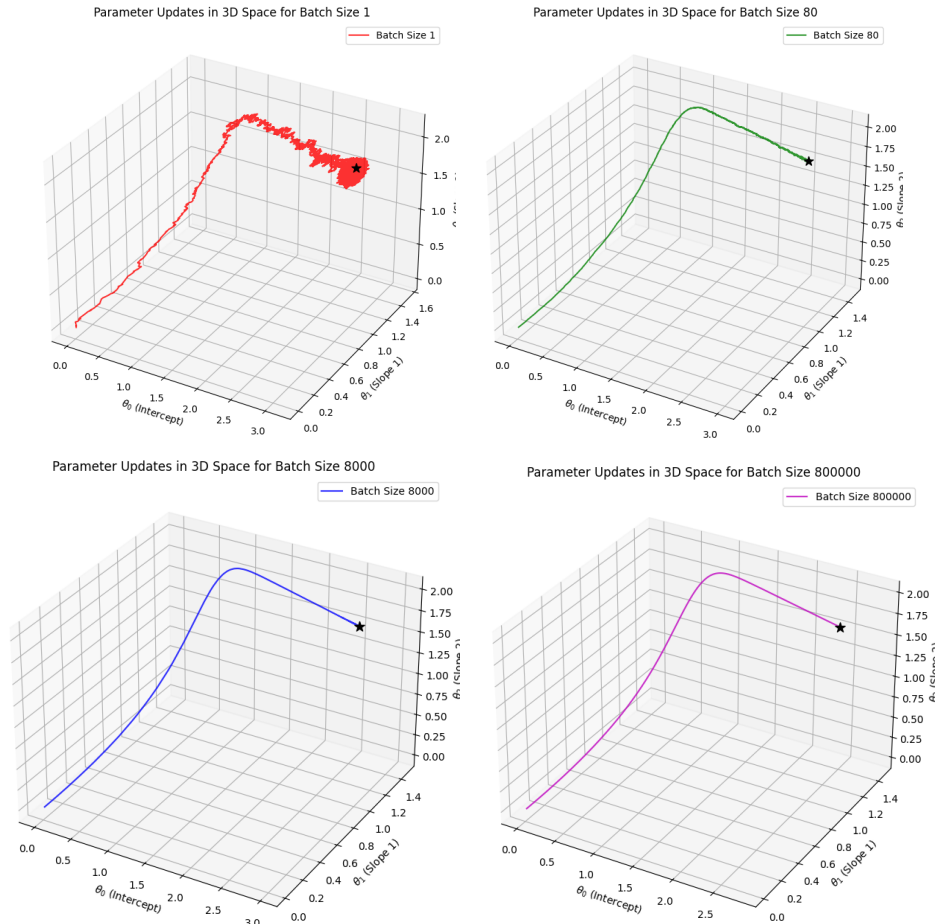
6) Small Batch ($r = 1$) Also Slightly Noisy

MSE is slightly higher than the closed-form, but still very close to 2.0. With many updates per epoch (80,000 updates), SGD can converge quickly but sometimes overshoot or fluctuate due to high variance gradients.

2.5. (4 points)

In the 3-dimensional parameter space (θ_0 on each axis), plot the movement of θ as the parameters are updated (until convergence) for varying batch sizes. How does the (shape of) movement compare in each case? Does it make intuitive sense? Argue.

Answer :



Observation and comments :

1) Small Batch Size (e.g., $r = 1$)

- Each iteration updates parameters using the gradient from a single sample.
- Due to high variance in individual sample gradients, the update path appears highly noisy and zigzagged.
- This noise causes the trajectory to fluctuate, making it less stable.
- However, since each sample contributes to an update, the algorithm performs 80,000 updates per epoch, leading to faster convergence in terms of epochs.
- While the high frequency of updates accelerates convergence, excessive noise can sometimes hinder reaching the global minimum optimally.

2) Moderate Batch Size (e.g., $r = 80$)

- Each update is computed using the gradient from a batch of 80 samples.
- The fluctuations are reduced because the gradient is now averaged over a larger subset of data, leading to a more stable descent path.

- The number of updates per epoch is now 1,000 (80,000 / 80), meaning that each epoch sees fewer updates than $r = 1$, but each update is more reliable.
- The tradeoff here is that while the parameter updates are less frequent, they are more accurate, striking a balance between noise and stability.

3) Larger Batch Size (e.g., $r = 8,000$)

- A batch consists of 8,000 samples, leading to 10 updates per epoch (80,000 / 8,000).
- The parameter update trajectory becomes much smoother, following a more direct path towards the optimal point.
- Since the gradient is estimated using a significant fraction of the dataset, noise is drastically reduced.
- However, the total number of updates is much lower per epoch, requiring more epochs to achieve convergence compared to smaller batch sizes.

4) Very Large Batch Size (e.g., $r = 80,000$)

- The entire dataset is used to compute a single gradient per epoch (i.e., 1 update per epoch).
- This results in an extremely smooth and direct trajectory following the steepest descent.
- However, since there is only one update per epoch, convergence is much slower compared to smaller batch sizes.
- While this approach provides the most precise gradient updates, it requires significantly more epochs to reach the minimum.

Key Takeaways

Small batches ($r = 1$) lead to faster but noisier convergence due to frequent updates.

Moderate batches ($r = 80, 8000$) provide a balance between noise and stability.

Very large batches ($r = 80,000$) yield the smoothest updates but require the most epochs to converge.

Mathematical Explanation of Batch Size Effects in SGD

Stochastic Gradient Descent (SGD) updates parameters θ using the gradient of the loss function $J(\theta)$ with respect to a batch of data samples:

$$\theta_{t+1} = \theta_t - \eta \nabla J_B(\theta_t)$$

where:

η is the learning rate.

$\nabla J_B(\theta_t)$ is the gradient estimate using a batch B of size r .

Effect of Different Batch Sizes

1. Small Batch Size ($r=1$):

The gradient is computed for a single data point, so it has a high variance:

$$\nabla J_1(\theta) \approx \mathbb{E}[\nabla J(\theta)] + \text{noise}$$

This leads to large fluctuations in the parameter updates. The number of updates per epoch = dataset size / 1=80,000.

2. Moderate Batch Size (r=80,r=8000):

The gradient is averaged over 80 or 8000 samples, reducing the noise. The update becomes more stable

$$\nabla J_B(\theta) = \frac{1}{r} \sum_{i=1}^r \nabla J_i(\theta)$$

Updates per epoch = 80,000 / 80 = 1,000 (for r=80) and 80,000 / 8,000=10 (for r=8000). This balances convergence speed and stability.

3. Very Large Batch Size (r=80,000):

The gradient is computed for the entire dataset in one go.

The update is smoothest and follows the exact steepest descent, but since there is only update per epoch, convergence is slow.

Updates per epoch = 80,000/80,000=1.

3 (15 points) Logistic Regression

3.1. (10 points)

The files `logisticX.csv` and `logisticY.csv` contain the inputs ($x(i) \in \mathbb{R}^2$) and outputs ($y(i) \in \{0,1\}$) respectively for a binary classification problem, with one training example per row. Implement [1](#) Newton's method for optimizing $L(\theta)$, and apply it to fit a logistic regression model to the data. Initialize Newton's method with $\theta = 0$ (the vector of all zeros). What are the coefficients θ resulting from your fit? (Remember to include the intercept term.)

Answer :

Logistic Regression is implemented in the file `logistic_regression.py` in Q3 folder. When running on the given `logisticX.csv` (normalised and intercept term added) and `logisticY.csv` I got θ as :

$$\theta = [0.40125316, 2.5885477, -2.72558849]$$

This result was obtained at a learning rate = 0.01 (As done in class Newtons Method do not have a learning rate as a parameter , but from the inference from the piazza post on the use of learning rate, I inferred that we are implementing a variation of Newtons method (quasi newtons method)) where θ was originally initialised to zero.

Convergence criteria : $\text{norm}(\text{delta_theta}) < 1e-7$

Max Iteration : $1e6$

Number of Iteration : 7

Logistic regression models the probability that a given input belongs to class 1 as follows:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

where θ is the parameter vector to be optimized.

Newton's method is an iterative approach used to find the maximum likelihood estimates of . The update rule for is given by:

$$\theta^{(t+1)} = \theta^{(t)} - H^{-1} \nabla L(\theta)$$

Where

- $\nabla L(\theta)$ is the gradient of the log-likelihood function.
- H is the Hessian matrix (second derivative of the log-likelihood).

The log-likelihood function for logistic regression is:

$$L(\theta) = \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

The gradient is computed as:

$$\nabla L(\theta) = \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

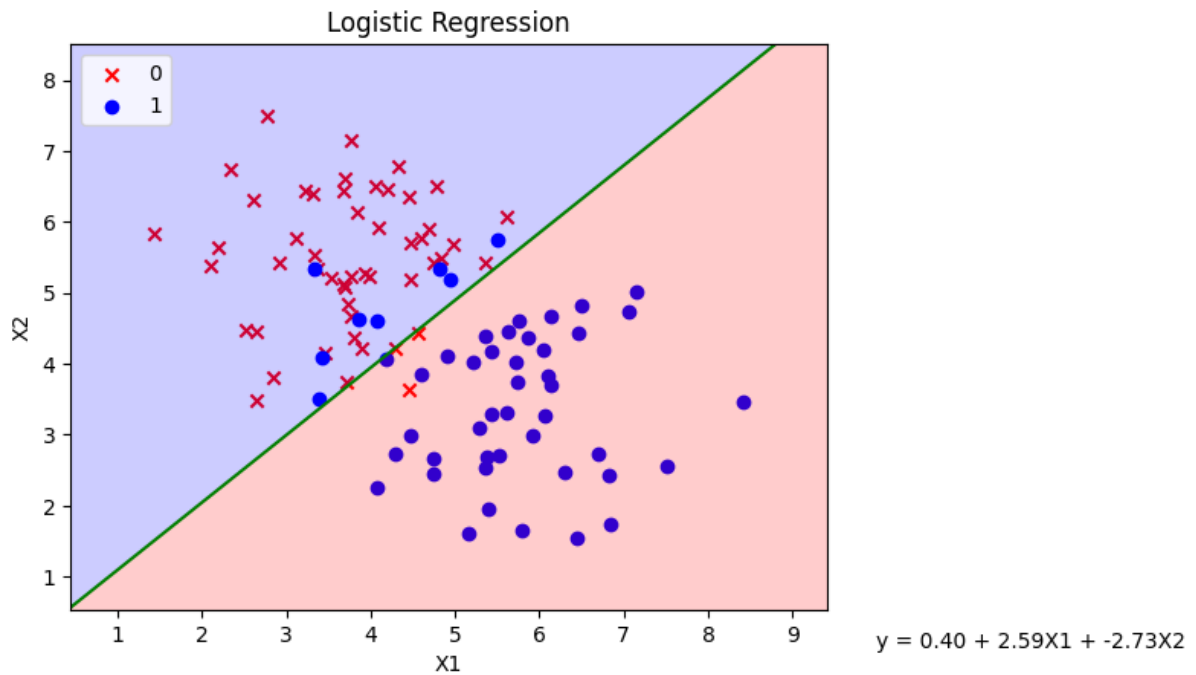
The Hessian matrix is given by:

$$H = \sum_{i=1}^m h_{\theta}(x^{(i)})(1 - h_{\theta}(x^{(i)})) x^{(i)} x^{(i)T}$$

3.2. (5 points)

Plot the training data (your axes should be x1 and x2 , corresponding to the two coordinates of the inputs, and you should use a different symbol for each point plotted to indicate whether that example had label 1 or 0). Also plot on the same figure the decision boundary fit by logistic regression. (i.e., this should be a straight line showing the boundary separating the region where $h(x) > 0.5$ from where $h(x) \leq 0.5$.)

Answer :



Accuracy on the same trained dataset : 0.88

4 (25 points) Gaussian Discriminant Analysis

4.1. (6 points)

Learn the Gaussian Discriminant Analysis parameters using the closed form equations described in class. Assume that both the classes have the same co-variance matrix i.e. $\Sigma_0 = \Sigma_1 = \Sigma$. Report the values of the means, μ_0 and μ_1 , and the co-variance matrix Σ .

Answer :

GDA is implemented in the file gda.py in Q4 folder. When running on the given q4x.dat and q4y.dat I got means, μ_0 and μ_1 , and the co-variance matrix Σ as :

$$\begin{aligned}\mu_0 &= [-0.75529433, 0.68509431] \\ \mu_1 &= [0.75529433, -0.68509431] \\ \Sigma &= \begin{bmatrix} 0.42953048 & -0.02247228 \\ -0.02247228 & 0.53064579 \end{bmatrix}\end{aligned}$$

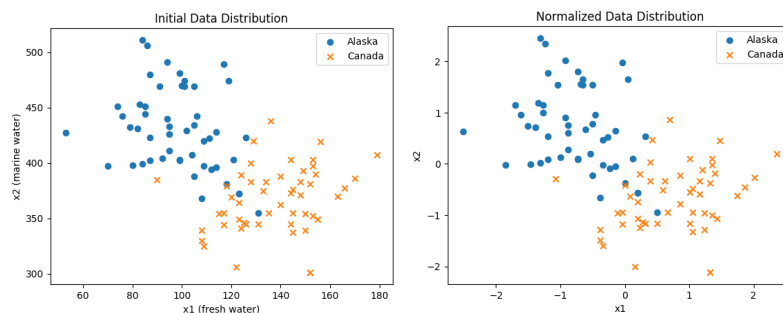
In GDA when I assume the same co-variance matrix :

$$\begin{aligned}\phi &= \frac{1}{m} \sum_{i=1}^m 1\{y^{(i)} = 1\} \\ \mu_0 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \\ \mu_1 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \\ \Sigma &= \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T.\end{aligned}$$

4.2. (2 points)

Plot the training data corresponding to the two coordinates of the input features, and you should use a different symbol for each point plotted to indicate whether that example had label Canada or Alaska.

Answer :



4.3. (3 points)

Describe the equation of the boundary separating the two regions in terms of the parameters μ_0, μ_1 and Σ . Recall that GDA results in a linear separator when the two classes have identical co-variance matrix. Along with the data points plotted in the part above, plot (on the same figure) decision boundary fit by GDA.

Answer :

If the covariance matrices of both classes are assumed to be equal, i.e.,

$$\Sigma_0 = \Sigma_1 = \Sigma$$

then the decision function simplifies to a linear form:

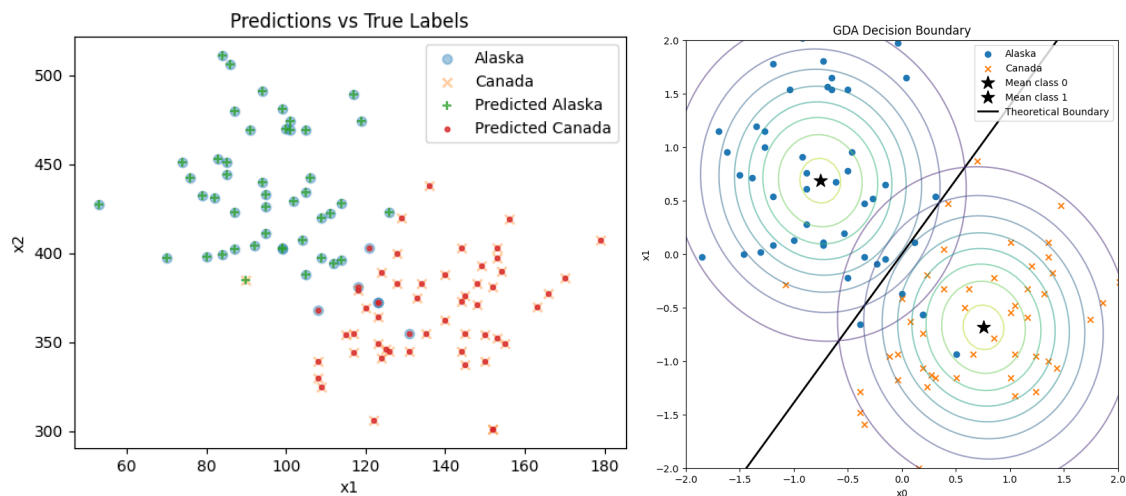
$$\delta(x) = x^T \Sigma^{-1}(\mu_1 - \mu_0) - \frac{1}{2}(\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} \mu_0) + \log \frac{P(y=1)}{P(y=0)}$$

which leads to a linear decision boundary:

$$w^T x + b = 0$$

Where

- $w = \Sigma^{-1}(\mu_1 - \mu_0)$,
- $b = -\frac{1}{2}(\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} \mu_0) + \log \frac{P(y=1)}{P(y=0)}$.



4.4. (6 points)

In general, GDA allows each of the target classes to have its own covariance matrix. This results (in general) results in a quadratic boundary separating the two class regions. In this case, the maximum-likelihood estimate of the co-variance matrix Σ_0 can be derived using the equation:

$$\Sigma_0 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T}{\sum_{i=1}^m 1\{y^{(i)} = 0\}}$$

And similarly, for Σ_1 . The expressions for the means remain the same as before. Implement GDA for the above problem in this more general setting. Report the values of the parameter estimates i.e. μ_0 , μ_1 , Σ_0 , Σ_1 .

Answer :

GDA is implemented in the file gda.py in Q4 folder. When running on the given q4x.dat and q4y.dat I got means, μ_0 and μ_1 , and the co-variance matrix Σ_1 and Σ_2 as :

$$\begin{aligned}\mu_0 &= [-0.75529433, 0.68509431] \\ \mu_1 &= [0.75529433, -0.68509431] \\ \Sigma_1 &= \begin{bmatrix} 0.38158978 & -0.15486516 \\ -0.15486516 & 0.64773717 \end{bmatrix} \\ \Sigma_2 &= \begin{bmatrix} 0.4774711 & 0.1099206 \\ 0.1099206 & 0.41355441 \end{bmatrix}\end{aligned}$$

4.5. (5 points)

Describe the equation for the quadratic boundary separating the two regions in terms of the parameters μ_0 , μ_1 and Σ_0 , Σ_1 . On the graph plotted earlier displaying the data points and the linear separating boundary, also plot the quadratic boundary obtained in the previous step.

Answer :

Quadratic Decision Boundary (QDA)

If the covariance matrices differ for each class, i.e.,

$$\Sigma_0 \neq \Sigma_1$$

then the decision function does not simplify linearly. Instead, it becomes:

$$\delta(x) = -\frac{1}{2}x^T \Sigma_1^{-1} x + x^T \Sigma_1^{-1} \mu_1 - \frac{1}{2} \mu_1^T \Sigma_1^{-1} \mu_1 - \frac{1}{2} \log |\Sigma_1|$$

$$+ \frac{1}{2} x^T \Sigma_0^{-1} x - x^T \Sigma_0^{-1} \mu_0 + \frac{1}{2} \mu_0^T \Sigma_0^{-1} \mu_0 + \frac{1}{2} \log |\Sigma_0| + \log \frac{P(y=1)}{P(y=0)}$$

which leads to a quadratic equation:

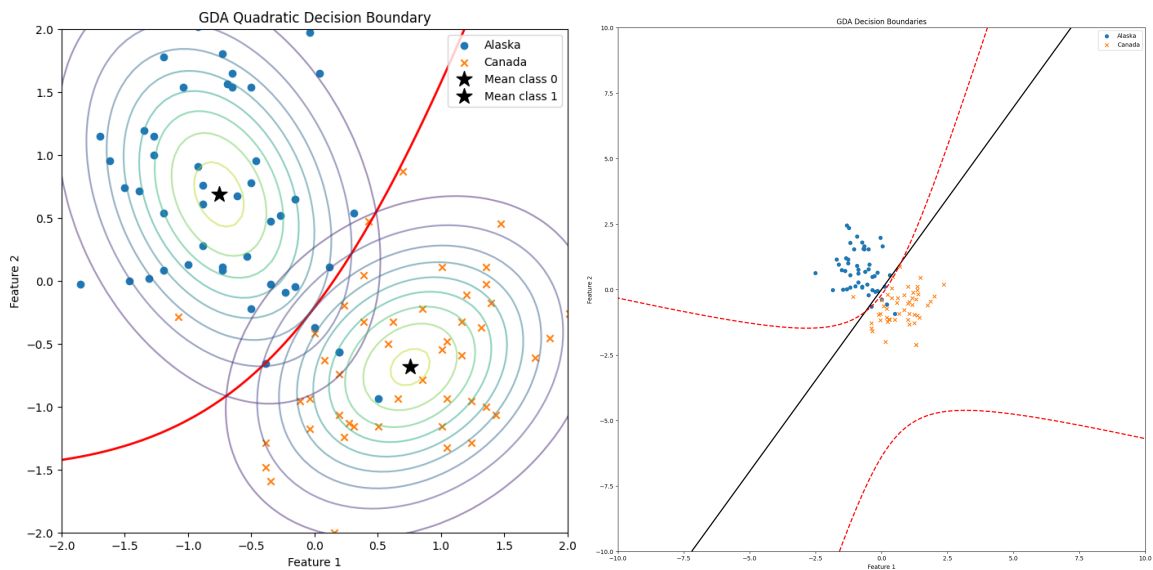
$$x^T A x + B^T x + C = 0$$

where:

- $A = \frac{1}{2}(\Sigma_0^{-1} - \Sigma_1^{-1})$,
- $B = \Sigma_1^{-1} \mu_1 - \Sigma_0^{-1} \mu_0$,
- $C = \text{constant terms}$.

Since this equation is quadratic in x , the decision boundary forms a curve (a quadratic surface like an ellipse, parabola, or hyperbola).

This explains why QDA results in curved decision boundaries, as seen in the images below



4.6. (3 points)

Carefully analyze the linear as well as the quadratic boundaries obtained. Comment on your observations.

Answer :

Observations:

On the test data the **accuracy** came out to be same as **0.93** , but on splitting the data into test and train, on both testing and training data the accuracy of QDA came out to be better than LDA for the given data.

1. Linear Decision Boundary (LDA)

LDA assumes that the covariance matrices of both classes are the same, leading to a linear separation.

The misclassified points near the boundary suggest that a linear model might not fully capture the complexity of class distributions.

2. Quadratic Decision Boundary (QDA)

The second and third images show QDA decision boundaries, which are curved.

Unlike LDA, QDA assumes that each class has its own covariance matrix, resulting in elliptical contours.

The contour ellipses in the plots indicate that the class distributions are better captured. The decision boundary is curved, adapting to the actual class distribution more effectively compared to the linear boundary.

Comparison between LDA and QDA :

Aspect	LDA (Linear)	QDA (Quadratic)
Assumption	Equal covariance	Different covariance
Decision boundary	Linear (straight line)	Quadratic (curve)
Model complexity	Lower (fewer parameters)	Higher (more parameters)
Required Data	Work well with fewer samples	Needs more data to estimate covariance matrices
Flexibility	Less flexible (good if data is well separated linearly)	More flexible (capture class variance better)

Conclusion :

- 1) LDA is optimal if class distributions have the same covariance structure (leading to a straight-line boundary).
- 2) QDA is more flexible and better if covariances differ (leading to curved boundaries that fit data better).
- 3) If training data is limited, LDA is preferable because QDA requires more data to estimate multiple covariance matrices accurately.